Cloud-Native DevOps Platform

PART-2

SEETHA LAKSHMI K A P

24-06-2026

Kubernetes (EKS) Configuration and Deployment Preparation

1)Objective

- Create Kubernetes manifests for staging and production.
- Configure Argo Rollouts using Canary deployment strategy.
- Configure External Secrets to retrieve secrets from AWS Secrets Manager.
- Configure AWS ALB Ingress.
- Prepare ArgoCD applications for GitOps deployment.

2) Kubernetes Files Created

2.1) cluster-secret-store.yaml

Location

`k8s/common/cluster-secret-store.yaml`

Purpose

This file configures the Cluster Secret Store, allowing the External Secrets Operator to retrieve secrets directly from AWS Secrets Manager.

Reason

Database credentials and sensitive information are securely managed in AWS Secrets Manager instead of being hardcoded in Kubernetes manifests.

2.2) namespace.yaml

Location

`k8s/staging/`
`k8s/prod/`

Purpose

Creates separate namespaces for the staging and production environments.

Reason

- Isolates resources between environments.
- Prevents conflicts between staging and production deployments.

2.3) external-secret.yaml

Location

```
k8s/staging/
k8s/prod/
```

Purpose

Retrieves database credentials from AWS Secrets Manager and creates Kubernetes Secrets automatically.

Secrets Retrieved

- Database Host
- Database Username
- Database Password

2.4) backend-rollout.yaml

Location

```
k8s/staging/
k8s/prod/
```

Purpose

Deploys the backend application using **Argo Rollouts**.

Important Configurations

- Amazon ECR backend image
- Replica count
- Resource requests and limits
- Environment variables
- Readiness probe
- Liveness probe
- **Canary deployment strategy**

Reason

Argo Rollouts enables controlled deployments with gradual traffic shifting and rollback support.

2.5) backend-service.yaml

Purpose

Creates a ClusterIP service for the backend application.

This service enables communication between the frontend and backend inside the Kubernetes cluster.

2.6) frontend-rollout.yaml

Purpose

Deploys the frontend application using Argo Rollouts.

Important Configurations

- Frontend image
- Replica count
- Resource limits
- Health checks
- Canary deployment strategy

2.7) frontend-service.yaml

Purpose

Creates the frontend ClusterIP service.

The service exposes the frontend pods internally, allowing the Ingress resource to route external traffic.

2.8) ingress.yaml

Purpose

Creates the Kubernetes Ingress resource.

The AWS Load Balancer Controller automatically provisions an Application Load Balancer (ALB) based on this manifest.

Functions

- Routes incoming HTTP requests.
- Connects the ALB with frontend services.
- Enables external access to the application.

3. Kubernetes Tool Installation

3.1) Configure kubectl

The Kubernetes CLI was configured to communicate with the Amazon EKS cluster.

Command:

```
aws eks update-kubeconfig --region ap-south-1 --name <cluster-name>
```

Verification:

```
kubectl get nodes
```

```
PS C:\Users\user\Downloads\task-5\terraform> aws eks update-kubeconfig --region us-west-1 --name task5-eks
Updated context arn:aws:eks:us-west-1:622488711452:cluster/task5-eks in C:\ProgramData\Jenkins\.kube\config
PS C:\Users\user\Downloads\task-5\terraform> kubectl get nodes
NAME                                          STATUS   ROLES    AGE   VERSION
ip-10-0-11-161.us-west-1.compute.internal     Ready    <none>   50m   v1.31.14-eks-93b80c6
ip-10-0-11-203.us-west-1.compute.internal     Ready    <none>   50m   v1.31.14-eks-93b80c6
ip-10-0-12-32.us-west-1.compute.internal      Ready    <none>   50m   v1.31.14-eks-93b80c6
ip-10-0-12-83.us-west-1.compute.internal      Ready    <none>   50m   v1.31.14-eks-93b80c6
```

3.2) Install Helm

Helm was used to install Kubernetes components required by the project.

Verify Installation:

```
helm version
```

3.3) Install Argo Rollouts

Argo Rollouts was installed to implement Canary deployments.

Create Namespace:

```
kubectl create namespace argo-rollouts
```

Install Controller:

```
kubectl apply -n argo-rollouts \
-f https://github.com/argoproj/argo-rollouts/releases/latest/download/install.yaml
```

Verification:

```
kubectl get pods -n argo-rollouts
```

```
kubectl argo rollouts version
```

```
PS C:\Users\user\Downloads\task-5\terraform> kubectl get pods -n argo-rollouts
NAME                             READY   STATUS    RESTARTS   AGE
argo-rollouts-54595797c6-6dqvc   1/1     Running   0          28s
PS C:\Users\user\Downloads\task-5\terraform>
```

3.4) Install External Secrets Operator

The External Secrets Operator was installed to synchronize secrets from AWS Secrets Manager.

Add Repository:

```
helm repo add external-secrets https://charts.external-secrets.io

helm repo update
```

Install:

```
helm install external-secrets external-secrets/external-secrets \
-n external-secrets \
--create-namespace
```

Verification

```
kubectl get pods -n external-secrets
```

```
PS C:\Users\user\Downloads\task-5> kubectl get pods -n external-secrets
NAME                                               READY   STATUS    RESTARTS   AGE
external-secrets-75f645766f-jtqjp                  1/1     Running   0          29s
external-secrets-cert-controller-5bbb8c7b6c-xjcdt  1/1     Running   0          29s
external-secrets-webhook-665f5f5699-5lcvk          1/1     Running   0          29s
PS C:\Users\user\Downloads\task-5>
```

3.5) Install AWS Load Balancer Controller

The AWS Load Balancer Controller was installed to provision Application Load Balancers from Kubernetes Ingress resources.

Add Helm Repository:

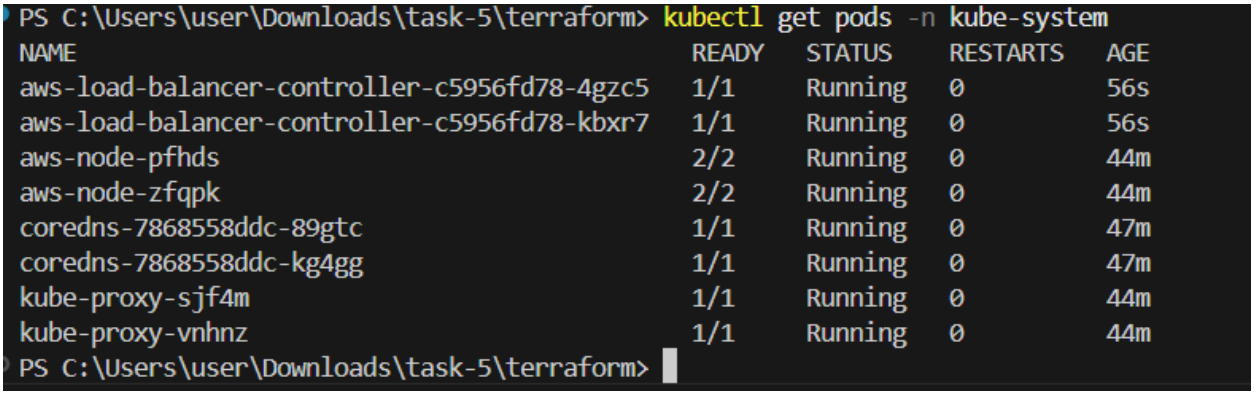
```
helm repo add eks https://aws.github.io/eks-charts

helm repo update
```

Install:

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
-n kube-system \
--set clusterName=<cluster-name> \
--set serviceAccount.create=false \
--set serviceAccount.name=aws-load-balancer-controller
```

Verification:

```
kubectl get deployment -n kube-system
```



4. Verification Commands

The following commands were used to verify the Kubernetes environment before enabling GitOps deployment:

- Check worker nodes : `kubectl get nodes`
- Check namespaces : `kubectl get namespaces`
- Check pods : `kubectl get pods -A`
- Check services : `kubectl get svc -A`
- Check Ingress resources : `kubectl get ingress -A`
- Check Cluster Secret Store : `kubectl get clustersecretstore`
- Check External Secrets : `kubectl get externalsecret -A`
- Check Argo Rollouts : `kubectl get rollouts -A`
- Describe a pod : `kubectl describe pod <pod-name>`
- View pod logs : `kubectl logs <pod-name>`

5. Outcome

During this phase, the Kubernetes environment was successfully prepared for GitOps deployment. Separate Kubernetes manifests were created for staging and production environments, Canary deployment was configured using Argo Rollouts, secrets were securely integrated through AWS Secrets Manager, and the AWS Load Balancer

Controller was configured to provision Application Load Balancers automatically. The Kubernetes cluster was verified and made ready for continuous deployment through ArgoCD.